

P1152R1

Deprecating **volatile**

Published Proposal, 2019-01-20

This version:

<http://wg21.link/P1152R1>

Author:

[JF Bastien](#) (Apple)

Audience:

SG1, LEWG, EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/P1152R1.bs

Table of Contents

- 1 **Abstract**
- 2 **Edit History**
 - 2.1 r0 → r1
- 3 **Wording**
 - 3.1 Program execution [**intro.execution**]
 - 3.2 Data races [**intro.races**]
 - 3.3 Forward progress [**intro.progress**]
 - 3.4 Class member access [**expr.ref**]
 - 3.5 The cv-qualifiers [**dcl.type.cv**]
 - 3.6 Functions [**dcl.fct**]
 - 3.7 Non-static member functions [**class.mfct.non-static**]
 - 3.8 The this pointer [**class.this**]

- 3.9 Constructors [**class.ctor**]
- 3.10 Destructors [**class.dtor**]
- 3.11 Overloadable declarations [**over.load**]
- 3.12 Built-in operators [**over.built**]
- 3.13 Tuples [**tuple**]
- 3.14 Variants [**variant**]
- 3.15 Atomic operations library [**atomics**]
- 3.16 Annex D

References

Informative References

§ 1. Abstract

We propose deprecating most of `volatile`. See [§3 Wording](#) for the details.

The proposed deprecation preserves the useful parts of `volatile`, and removes the dubious / already broken ones. This paper aims at breaking at compile-time code which is today subtly broken at runtime or through a compiler update. The paper might also break another type of code: that which doesn't exist. This removes a significant foot-gun and removes unintuitive corner cases from the languages.

The first version of this paper, [\[P1152R0\]](#), has extensive background information which is not repeated here:

- [Overview](#)
- [How we got here](#)
- [Why the proposed changes?](#)
- [Examples](#)

See [\[P1382R0\]](#) for the follow-up paper on `volatile_load<T>` / `volatile_store<T>` requested by SG1.

§ 2. Edit History

§ 2.1. r0 → r1

[P1152R0] was seen by SG1 and EWG in San Diego. This update does the following:

- Remove background information from the paper.
- Follow the guidance from SG1 and EWG, based on the polls below.

Poll	Group	SF	F	N	A	SA	Outcome
Deprecate volatile compound operations (including ++ and --) on scalar types (arithmetic, pointer, enumeration).	SG1	4	19	3	0	0	✓
Deprecate volatile compound operations (including ++ and --) on scalar types (arithmetic, pointer, enumeration).	EWG	4	9	4	0	0	✓
Deprecate usage of volatile assignment chaining on scalar types (arithmetic, pointer, enumeration, pointer to members, nullptr_t).	SG1	6	15	3	0	0	✓
Deprecate usage of volatile assignment chaining on scalar types (arithmetic, pointer, enumeration, pointer to members, nullptr_t).	EWG	6	9	3	0	0	✓
SG1 would be OK if we deprecated volatile qualified member functions (pending separate decision on what we do with volatile atomic).	SG1	1	5	10	4	3	✗
EWG would be OK if we deprecated volatile qualified member functions (pending separate decision on what we do with volatile atomic).	EWG	2	7	7	1	0	✓
SG1 would be OK if we deprecated volatile partial template specializations, overloads, or qualified member functions in the STL for all but the atomic, numeric_limits , and type traits (remove_volatile , add_volatile , etc) parts of the Library.	SG1	1	9	6	2	0	✓

Poll	Group	SF	F	N	A	SA	Outcome
EWG would be OK if we deprecated volatile partial template specializations, overloads, or qualified member functions in the STL for all but the atomic, numeric_limits , and type traits (remove_volatile , add_volatile , etc) parts of the Library.	EWG	1	11	9	0	0	✓
Deprecate volatile member functions of atomic in favor of new template partial specializations which will only declare load, store, and only exist when is_always_lock_free is true.	SG1	2	1	1	11	2	✗
Deprecate volatile member functions of atomic in favor of new template partial specializations which will only declare load, store, RMW, and only exist when is_always_lock_free is true.	SG1	4	7	3	3	0	✓
Deprecate volatile member functions of atomic in favor of new template partial specializations which will only declare load, store, RMW, and only exist when is_always_lock_free is true.	EWG	2	9	3	0	0	✓
Deprecate volatile member functions of atomic in favor of new template partial specializations which will only declare load, store, RMW.	SG1	0	0	0	10	7	✗
SG1 would be OK if we deprecated top-level volatile parameters.	SG1	6	9	6	2	1	✓
EWG would be OK if we deprecated top-level volatile parameters.	EWG	6	9	6	0	0	✓
EWG would be OK if we deprecated top-level const parameters.	EWG	0	2	5	8	8	✗
SG1 would be OK if we deprecated top-level volatile return values.	SG1	6	9	4	2	0	✓
EWG would be OK if we deprecated top-level volatile return values.	EWG	6	6	5	0	0	✓

Poll	Group	SF	F	N	A	SA	Outcome
EWG would be OK if we deprecated top-level const return values.	EWG	2	3	3	5	5	✗
SG1 interested is interested in hearing about <code>volatile_load<T> /</code> <code>volatile_store<T></code> free functions in a separate paper, given that time is limited and we could be doing something else.	SG1	0	17	4	3	0	✓
EWG interested is interested in hearing about <code>volatile_load<T> /</code> <code>volatile_store<T></code> free functions in a separate paper, given that time is limited and we could be doing something else.	EWG	2	11	4	1	0	✓

§ 3. Wording

§ 3.1. Program execution [intro.execution]

No changes.

Accesses through `volatile` glvalues are evaluated strictly according to the rules of the abstract machine.

Reading an object designated by a `volatile` glvalue, modifying an object, calling a library I/O function, or calling a function that does any of those operations are all *side effects*, which are changes in the state of the execution environment. *Evaluation* of an expression (or a subexpression) in general includes both value computations (including determining the identity of an object for glvalue evaluation and fetching a value previously assigned to an object for prvalue evaluation) and initiation of side effects. When a call to a library I/O function returns or an access through a `volatile` glvalue is evaluated the side effect is considered complete, even though some external actions implied by the call (such as the I/O itself) or by the `volatile` access may not have completed yet.

§ 3.2. Data races [intro.races]

No changes.

Two accesses to the same object of type `volatile std::sig_atomic_t` do not result in a data race if both occur in the same thread, even if one or more occurs in a signal handler. For each signal handler invocation, evaluations performed by the thread invoking a signal handler can be divided into two groups A and B, such that no evaluations in B happen before evaluations in A, and the evaluations of such `volatile std::sig_atomic_t` objects take values as though all evaluations in A happened before the execution of the signal handler and the execution of the signal handler happened before all evaluations in B.

§ 3.3. Forward progress [intro.progress]

No changes.

The implementation may assume that any thread will eventually do one of the following:

- terminate,
- make a call to a library I/O function,
- perform an access through a `volatile` glvalue, or
- perform a synchronization operation or an atomic operation

During the execution of a thread of execution, each of the following is termed an execution step:

- termination of the thread of execution,
- performing an access through a `volatile` glvalue, or
- completion of a call to a library I/O function, a synchronization operation, or an atomic operation.

§ 3.4. Class member access [expr.ref]

No changes.

Abbreviating *postfix-expression.id-expression* as `E1.E2`, `E1` is called the *object expression*. If `E2` is a bit-field, `E1.E2` is a bit-field. The type and value category of `E1.E2` are determined as follows. In the remainder of [expr.ref], *cq* represents either `const` or the absence of `const` and *vq* represents either `volatile` or the absence of `volatile`. *cv* represents an arbitrary set of cv-qualifiers.

- If `E2` is a non-static data member and the type of `E1` is “*cq1 vq1 X*”, and the type of `E2` is “*cq2 vq2 T*”, the expression designates the named member of the object designated by the first expression. If `E1` is an lvalue, then `E1.E2` is an lvalue; otherwise `E1.E2` is an xvalue. Let the notation *vq12* stand for the “union” of *vq1* and *vq2*; that is, if *vq1* or *vq2* is `volatile`, then *vq12* is `volatile`. Similarly, let the notation *cq12* stand for the “union” of *cq1* and *cq2*; that is, if *cq1* or *cq2* is `const`, then *cq12* is `const`. If `E2` is declared to be a `mutable` member, then the type of `E1.E2` is “*vq12 T*”. If `E2` is not declared to be a `mutable` member, then the type of `E1.E2` is “*cq12 vq12 T*”.

§ 3.5. The cv-qualifiers [dcl.type.cv]

No changes.

The semantics of an access through a `volatile` glvalue are implementation-defined. If an attempt is made to access an object defined with a `volatile`-qualified type through the use of a non-`volatile` glvalue, the behavior is undefined.

[*Note*: `volatile` is a hint to the implementation to avoid aggressive optimization involving the object because the value of the object might be changed by means undetectable by an implementation. Furthermore, for some implementations, `volatile` might indicate that special hardware instructions are required to access the object. See [intro.execution] for detailed semantics. In general, the semantics of `volatile` are intended to be the same in C++ as they are in C. —end note]

§ 3.6. Functions [dcl.fct]

Modify as follows.

The *parameter-declaration-clause* determines the arguments that can be specified, and their processing, when the function is called. [*Note*: The *parameter-declaration-clause* is used to convert the arguments specified on the function call; see [expr.call] —end note] If the *parameter-declaration-clause* is empty, the function takes no arguments. A parameter list consisting of a single unnamed parameter of non-dependent type `void` is equivalent to an empty parameter list. Except for this special case, a parameter shall not have type cv `void`. A parameter's declarator shall only allow `const` as its parameters-and-qualifiers's cv-qualifier-seq. If the *parameter-declaration-clause* terminates with an ellipsis or a function parameter pack, the number of arguments shall be equal to or greater than the number of parameters that do not have a default argument and are not function parameter packs. Where syntactically correct and where "... " is not part of an *abstract-declarator*, "`, ...`" is synonymous with "...".

[...]

The type of a function is determined using the following rules. The type of each parameter (including function parameter packs) is determined from its own *decl-specifier-seq* and *declarator*. After determining the type of each parameter, any parameter of type "array of `T`" or of function type `T` is adjusted to be "pointer to `T`". After producing the list of parameter types, any top-level *cv-qualifiers* modifying a parameter type are deleted when forming the function type. The resulting list of transformed parameter types and the presence or absence of the ellipsis or a function parameter pack is the function's *parameter-type-list*.

[...]

Functions shall not have a return type of type array or function, although they may have a return type of type pointer or reference to such things. There shall be no arrays of functions, although there can be arrays of pointers to functions.

Functions shall not have a `volatile` qualified return type.

§ 3.7. Non-static member functions [class.mfct.non-static]

No changes.

A non-static member function may be declared `const`, `volatile`, or `const volatile`. These cv-qualifiers affect the type of the `this` pointer. They also affect the function type of the member function; a member function declared `const` is a `const` member function, a member function declared `volatile` is a `volatile` member function and a member function declared `const volatile` is a `const volatile` member function.

§ 3.8. The `this` pointer [`class.this`]

No changes.

In the body of a non-static member function, the keyword `this` is a prvalue expression whose value is the address of the object for which the function is called. The type of `this` in a member function of a class `X` is `X*`. If the member function is declared `const`, the type of `this` is `const X*`, if the member function is declared `volatile`, the type of `this` is `volatile X*`, and if the member function is declared `const volatile`, the type of `this` is `const volatile X*`.

`volatile` semantics apply in `volatile` member functions when accessing the object and its non-static data members.

§ 3.9. Constructors [`class.ctor`]

No changes.

A constructor can be invoked for a `const`, `volatile` or `const volatile` object. `const` and `volatile` semantics are not applied on an object under construction. They come into effect when the constructor for the most derived object ends.

§ 3.10. Destructors [`class.dtor`]

No changes.

A destructor is used to destroy objects of its class type. The address of a destructor shall not be taken. A destructor can be invoked for a `const`, `volatile` or `const volatile` object. `const` and `volatile` semantics are not applied on an object under destruction. They stop being in effect when the destructor for the most derived object starts.

§ 3.11. Overloadable declarations [`over.load`]

Modify as follows.

Parameter declarations that differ only in the presence or absence of `const` and/or `volatile` are equivalent. That is, the `const` and `volatile` type-specifiers for each parameter type are ignored when determining which function is being declared, defined, or called.

§ 3.12. Built-in operators [**over.built**]

Modify as follows.

In the remainder of this section, ~~vq~~ represents either ~~volatile~~ or no cv-qualifier.

For every ~~pair~~ (T, vq) , where ~~T is an~~ arithmetic type T other than `bool`, there exist candidate operator functions of the form

```
vq T & operator++(vq T &);  
T operator++(vq T &, int);
```

For every ~~pair~~ (T, vq) , where ~~T is an~~ arithmetic type T other than `bool`, there exist candidate operator functions of the form

```
vq T & operator--(vq T &);  
T operator--(vq T &, int);
```

For every ~~pair~~ (T, vq) T , where T is a cv-qualified or cv-unqualified object type, there exist candidate operator functions of the form

```
T*vq & operator++(T*vq &);  
T*vq & operator--(T*vq &);  
T* operator++(T*vq &, int);  
T* operator--(T*vq &, int);
```

For every quintuple $(C1, C2, T, \text{cv1}, \text{cv2})$, where $C2$ is a class type, $C1$ is the same type as $C2$ or is a derived class of $C2$, and T is an object type or a function type, there exist candidate operator functions of the form

```
cv12 T& operator->*(cv1 C1*, cv2 T C2::*);
```

For every ~~triple~~ ~~pair~~ (L, vq, R) , where L is an arithmetic type, and R is a promoted arithmetic type, there exist candidate operator functions of the form

```

void operator=(volatile L&, R);
vqL& operator=(vqL&, R);
vqL& operator*=(vqL&, R);
vqL& operator/=(vqL&, R);
vqL& operator+=(vqL&, R);
vqL& operator-=(vqL&, R);

```

For ~~every pair~~ (T, vq) , where T is any type all types T , there exist candidate operator functions of the form

```

void operator=(T*volatile&, T*);
T*vq& operator=(T*vq&, T*);

```

For every ~~pair~~ (T, vq) , where T is an enumeration or pointer to member type T , there exist candidate operator functions of the form

```

void operator=(volatile T&, T);
vqT& operator=(vqT&, T);

```

For every ~~pair~~ (T, vq) T , where T is a cv-qualified or cv-unqualified object type, there exist candidate operator functions of the form

```

T*vq& operator+=(T*vq&, std::ptrdiff_t);
T*vq& operator-=(T*vq&, std::ptrdiff_t);

```

For every ~~triple pair~~ (L, vq, R) , where L is an integral type, and R is a promoted integral type, there exist candidate operator functions of the form

```
vq L& operator%=(vq L&, R);  
vq L& operator<=<=(vq L&, R);  
vq L& operator>=>=(vq L&, R);  
vq L& operator&=(vq L&, R);  
vq L& operator^=(vq L&, R);  
vq L& operator|=(vq L&, R);
```

§ 3.13. Tuples [tuple]

Modify as follows.

Header `<tuple>` synopsis [**tuple.syn**]:

```
namespace std {  
  
    [...]   
  
    // [tuple.helper, tuple helper classes  
    template<class T> class tuple_size; // not defined  
    template<class T> class tuple_size<const T>;  
template<class T> class tuple_size<volatile T>;  
template<class T> class tuple_size<const volatile T>;  
  
    template<class... Types> class tuple_size<tuple<Types...>>;  
  
    template<size_t I, class T> class tuple_element; // not defined  
    template<size_t I, class T> class tuple_element<I, const T>;  
template<size_t I, class T> class tuple_element<I, volatile T>;  
template<size_t I, class T> class tuple_element<I, const volatile T>;  
  
    [...]   
  
}
```

[...]

Tuple helper classes [**tuple.helper**]

```
template<class T> class tuple_size<const T>;  
template<class T> class tuple_size<volatile T>;  
  
template<class T> class tuple_size<const volatile T>;
```

Let `TS` denote `tuple_size<T>` of the cv-unqualified type `T`. If the expression `TS::value` is well-formed when treated as an unevaluated operand, then each of the three templates shall satisfy

the `TransformationTrait` requirements with a base characteristic of

```
integral_constant<size_t, TS::value>
```

Otherwise, they shall have no member `value`.

Access checking is performed as if in a context unrelated to `TS` and `T`. Only the validity of the immediate context of the expression is considered. [*Note*: The compilation of the expression can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the "immediate context" and can result in the program being ill-formed. —*end note*]

In addition to being available via inclusion of the `<tuple>` header, the ~~three templates are~~ template is available when any of the headers `<array>`, `<ranges>`, or `<utility>` are included.

```
template<size_t I, class T> class tuple_element<I, const T>;  
template<size_t I, class T> class tuple_element<I, volatile T>;  
template<size_t I, class T> class tuple_element<I, const volatile T>;
```

Let `TE` denote `tuple_element_t<I, T>` of the cv-unqualified type `T`. Then ~~each of the three templates~~ the template shall satisfy the `TransformationTrait` requirements with a member typedef `type` that names the following type: `add_const_t<TE>`.

- ~~for the first specialization, `add_const_t<TE>`;~~
- ~~for the second specialization, `add_volatile_t<TE>`, and~~
- ~~for the third specialization, `add_cv_t<TE>`;~~

In addition to being available via inclusion of the `<tuple>` header, the ~~three templates are~~ template is available when any of the headers `<array>`, `<ranges>`, or `<utility>` are included.

§ 3.14. Variants [**variant**]

Modify as follows.

`<variant>` synopsis [`variant.syn`]

```
namespace std {
// [variant.variant], class template variant
template<class... Types>
    class variant;

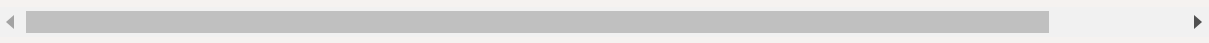
// [variant.helper], variant helper classes
template<class T> struct variant_size; // not defir
template<class T> struct variant_size<const T>;
template<class T> struct variant_size<volatile T>;
template<class T> struct variant_size<const volatile T>;
template<class T>
    inline constexpr size_t variant_size_v = variant_size<T>::value;

template<class... Types>
    struct variant_size<variant<Types...>>;

template<size_t I, class T> struct variant_alternative; // not defir
template<size_t I, class T> struct variant_alternative<I, const T>;
template<size_t I, class T> struct variant_alternative<I, volatile T>;
template<size_t I, class T> struct variant_alternative<I, const volatile T>;

[... ]

}
```



`variant` helper classes [`variant.helper`]

```
template<class T> struct variant_size;
```

Remark: All specializations of `variant_size` shall satisfy the `UnaryTypeTrait` requirements with a base characteristic of `integral_constant<size_t, N>` for some `N`.

```
template<class T> class variant_size<const T>;
template<class T> class variant_size<volatile T>;
template<class T> class variant_size<const volatile T>;
```

Let `VS` denote `variant_size<T>` of the cv-unqualified type `T`. Then ~~each of the three templates~~ the template shall satisfy the `UnaryTypeTrait` requirements with a base characteristic of `integral_constant<size_t, VS::value>`.

```
template<class... Types>
    struct variant_size<variant<Types...>> : integral_constant<size_t,
```

```
template<size_t I, class T> class variant_alternative<I, const T>;
template<size_t I, class T> class variant_alternative<I, volatile T>;
template<size_t I, class T> class variant_alternative<I, const volatile T>;
```

Let `VA` denote `variant_alternative<I, T>` of the cv-unqualified type `T`. Then ~~each of the three templates~~ the template shall meet the `TransformationTrait` requirements with a member typedef `type` that names the following type: `add_const_t<VA::type>`.

- for the first specialization, `add_const_t<VA::type>`,
- for the second specialization, `add_volatile_t<VA::type>`, and
- for the third specialization, `add_cv_t<VA::type>`.

§ 3.15. Atomic operations library [**atomics**]

Modify as follows.

Operations on atomic types [atomics.types.operations]

[*Note*: Many operations are volatile-qualified. The "volatile as device register" semantics have not changed in the standard. This qualification means that volatility is preserved when applying these operations to volatile objects. It does not mean that operations on non-volatile objects become volatile. —*end note*]

[...]

```
bool is_lock_free() const volatile noexcept;  
bool is_lock_free() const noexcept;
```

Returns: `true` if the object's operations are lock-free, `false` otherwise.

[*Note*: The return value of the `is_lock_free` member function is consistent with the value of `is_always_lock_free` for the same type. —*end note*]

```
void store(T desired, memory_order order = memory_order::seq_cst) vol  
void store(T desired, memory_order order = memory_order::seq_cst) noe
```

Requires: The `order` argument shall not be `memory_order::consume`, `memory_order::acquire`, nor `memory_order::acq_rel`.

Effects: Atomically replaces the value pointed to by `this` with the value of `desired`. Memory is affected according to the value of `order`.

Remarks: The `volatile` overload shall only participate in overload resolution when `atomic<T>::is_always_lock_free` is `true`.

```
T operator=(T desired) volatile noexcept;  
T operator=(T desired) noexcept;
```

Effects: Equivalent to `store(desired)`.

Returns: `desired`.

```
T load(memory_order order = memory_order::seq_cst) const volatile noexcept;
T load(memory_order order = memory_order::seq_cst) const noexcept;
```

Requires: The `order` argument shall not be `memory_order::release` nor `memory_order::acq_rel`.

Effects: Memory is affected according to the value of `order`.

Returns: Atomically returns the value pointed to by `this`.

Remarks: The `volatile` overload shall only participate in overload resolution when `atomic<T>::is_always_lock_free` is `true`.

```
operator T() const volatile noexcept;
operator T() const noexcept;
```

Effects: Equivalent to: `return load();`

```
T exchange(T desired, memory_order order = memory_order::seq_cst) volatile;
T exchange(T desired, memory_order order = memory_order::seq_cst) noexcept;
```

Effects: Atomically replaces the value pointed to by `this` with `desired`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations.

Returns: Atomically returns the value pointed to by `this` immediately before the effects.

Remarks: The `volatile` overload shall only participate in overload resolution when `atomic<T>::is_always_lock_free` is `true`.

```

bool compare_exchange_weak(T& expected, T desired,
                           memory_order success, memory_order failure)
bool compare_exchange_weak(T& expected, T desired,
                           memory_order success, memory_order failure)
bool compare_exchange_strong(T& expected, T desired,
                             memory_order success, memory_order failure)
bool compare_exchange_strong(T& expected, T desired,
                             memory_order success, memory_order failure)
bool compare_exchange_weak(T& expected, T desired,
                           memory_order order = memory_order::seq_cst)
bool compare_exchange_weak(T& expected, T desired,
                           memory_order order = memory_order::seq_cst)
bool compare_exchange_strong(T& expected, T desired,
                             memory_order order = memory_order::seq_cst)
bool compare_exchange_strong(T& expected, T desired,
                             memory_order order = memory_order::seq_cst)

```

Requires: The `failure` argument shall not be `memory_order::release` nor `memory_order::acq_rel`.

Effects: Retrieves the value in `expected`. It then atomically compares the value representation of the value pointed to by `this` for equality with that previously retrieved from `expected`, and if true, replaces the value pointed to by `this` with that in `desired`. If and only if the comparison is true, memory is affected according to the value of `success`, and if the comparison is false, memory is affected according to the value of `failure`. When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`. If and only if the comparison is false then, after the atomic operation, the value in `expected` is replaced by the value pointed to by `this` during the atomic comparison. If the operation returns `true`, these operations are atomic read-modify-write operations on the memory pointed to by `this`. Otherwise, these operations are atomic load operations on that memory.

Returns: The result of the comparison.

Remarks: The `volatile` overloads shall only participate in overload resolution when `atomic<T>::is_always_lock_free` is true.

[...]

Specializations for integers [**atomics.types.int**]

```
T fetch_key(T operand, memory_order order = memory_order::seq_cst) volatile  
T fetch_key(T operand, memory_order order = memory_order::seq_cst) noexcept
```

Effects: Atomically replaces the value pointed to by **this** with the result of the computation applied to the value pointed to by **this** and the given **operand**. Memory is affected according to the value of **order**. These operations are atomic read-modify-write operations.

Returns: Atomically, the value pointed to by **this** immediately before the effects.

Remarks: The **volatile** overloads shall only participate in overload resolution when **atomic<T>::is_always_lock_free** is true.

Remarks: For signed integer types, the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type. [*Note:* There are no undefined results arising from the computation. —end note]

```
T operator op=(T operand) volatile noexcept;  
T operator op=(T operand) noexcept;
```

Effects: Equivalent to: **return fetch_key(operand) op operand;**

Remarks: The **volatile** overloads shall only participate in overload resolution when **atomic<T>::is_always_lock_free** is true.

Specializations for floating-point types [**atomics.types.float**]

The following operations perform arithmetic addition and subtraction computations. The key, operator, and computation correspondence are identified in [**atomic.arithmetic.computations**].

```
T A::fetch_key(T operand, memory_order order = memory_order_seq_cst)
T A::fetch_key(T operand, memory_order order = memory_order_seq_cst)
```

Effects: Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations.

Returns: Atomically, the value pointed to by `this` immediately before the effects.

Remarks: The `volatile` overloads shall only participate in overload resolution when `atomic<T>::is_always_lock_free` is `true`.

Remarks: If the result is not a representable value for its type the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on `floating-point` should conform to the `std::numeric_limits<floating-point>` traits associated with the floating-point type. The floating-point environment for atomic arithmetic operations on `floating-point` may be different than the calling thread's floating-point environment.

```
T operator op=(T operand) volatile noexcept;
T operator op=(T operand) noexcept;
```

Effects: Equivalent to: `return fetch_key(operand) op operand;`

Remarks: If the result is not a representable value for its type the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on `floating-point` should conform to the `std::numeric_limits<floating-point>` traits associated with the floating-point type. The floating-point environment for atomic arithmetic operations on `floating-point` may be different than the calling thread's floating-point environment.

Partial specialization for pointers [`atomics.types.pointer`]

```
T* fetch_key(ptrdiff_t operand, memory_order order = memory_order::seq
T* fetch_key(ptrdiff_t operand, memory_order order = memory_order::seq
```

Requires: T shall be an object type, otherwise the program is ill-formed. [*Note:* Pointer arithmetic on `\tcode{void}` or function pointers is ill-formed. —end note*]

Effects: Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations.

Returns: Atomically, the value pointed to by `this` immediately before the effects.

Remarks: The `volatile` overloads shall only participate in overload resolution when `atomic<T>::is_always_lock_free` is true.

Remarks: The result may be an undefined address, but the operations otherwise have no undefined behavior.

```
T* operator op=(ptrdiff_t operand) volatile noexcept;
T* operator op=(ptrdiff_t operand) noexcept;
```

Effects: Equivalent to: `return fetch_key(operand) op operand;`

Member operators common to integers and pointers to objects [**atomics.types.memop**]

```
T operator++(int) volatile noexcept;
T operator++(int) noexcept;
```

Effects: Equivalent to: `return fetch_add(1);`


```
T operator--(int) volatile noexcept;  
T operator--(int) noexcept;
```

Effects: Equivalent to: `return fetch_sub(1);`

```
T operator++() volatile noexcept;  
T operator++() noexcept;
```

Effects: Equivalent to: `return fetch_add(1) + 1;`

```
T operator--() volatile noexcept;  
T operator--() noexcept;
```

Effects: Equivalent to: `return fetch_sub(1) - 1;`

Non-member functions [**atomics.nonmembers**]

A non-member function template whose name matches the pattern `atomic_f` or the pattern `atomic_f_explicit` invokes the member function `f`, with the value of the first parameter as the object expression and the values of the remaining parameters (if any) as the arguments of the member function call, in order. An argument for a parameter of type `atomic<T>::value_type*` is dereferenced when passed to the member function call. If no such member function exists, the program is ill-formed.

```
template<class T>  
    void atomic_init(volatile atomic<T>* object, typename atomic<T>::value_type  
template<class T>  
    void atomic_init(atomic<T>* object, typename atomic<T>::value_type
```

Effects: Non-atomically initializes `*object` with value `desired`. This function shall only be applied to objects that have been default constructed, and then only once. [*Note:* These semantics ensure compatibility with C. —*end note*] [*Note:* Concurrent access from another thread, even via an atomic operation, constitutes a data race. —*end note*]

Remarks: The `volatile` overloads shall only participate in overload resolution when `atomic<T>::is_always_lock_free` is true.

[*Note:* The non-member functions enable programmers to write code that can be compiled as either C or C++, for example in a shared header file. —*end note*]

§ 3.16. Annex D

All deletions above should be added to Annex D, such that `volatile` is now deprecated in these use cases.

§ References

§ Informative References

[P1152R0]

JF Bastien. [Deprecating volatile](https://wg21.link/p1152r0). 1 October 2018. URL: <https://wg21.link/p1152r0>

[P1382R0]

Paul Mckenney; JF Bastien. [volatile_load<T> and volatile_store<T>](http://wg21.link/P1382R0). URL: <http://wg21.link/P1382R0>